

Program Correctness: Mechanics

When examining the detail of the algorithm, it seems probable that the proof will be helpful in explaining not only what is happening but why.

— Tony Hoare

An Axiomatic Basis for Computer Programming, 1969

We are finally ready to apply FOL and induction to a real problem: specifying and proving properties of programs. In this chapter, we develop the three foundational methods that underly all verification and program analysis techniques. In the next chapter, we discuss strategies for applying them.

First, **specification** is the precise statement of properties that a program should exhibit. The language of FOL offers precision. The remaining task is to develop a scheme for embedding FOL statements into program text as **program annotations**. We focus on two forms of properties. **Partial correctness** properties, or **safety** properties, assert that certain states — typically, error states — cannot ever occur during the execution of a program. An important subset of this form of property is the partial correctness of programs: *if* a program halts, then its output satisfies some relation with its input. **Total correctness** properties, or **progress** properties, assert that certain states are eventually reached during program execution. Section 5.1 presents specification in the context of a simple programming language, `pi`.

The next foundational method is the **inductive assertion method** for proving partial correctness properties. The inductive assertion method is based on the mathematical induction of Chapter 4. To prove that every state during the execution of a program satisfies FOL formula F , prove as the base case that F holds at the beginning of execution; assume as the inductive hypothesis that F currently holds (at some point during the execution); and prove as the inductive step that F holds after one more step of the program. Section 5.2 discusses the mechanics for reducing a program with a partial correctness specification to this inductive argument. The challenge in applying this method is to discover additional annotations to make the induction go through. Chapter 6 discusses strategies for finding the extra information.

The third foundational method is the **ranking function method** for proving total correctness properties. Proving total correctness breaks down into two arguments. First, one proves that some partial correctness property holds using the inductive assertion method; second, one argues that some set of loops and recursive functions always halt. The ranking function method applies to the latter argument: one associates with each loop and recursive function a **ranking function** that maps the program variables to a well-founded domain (see Chapter 4); then one proves that whenever program control moves from one ranking function to the next, the value decreases according to the well-founded relation. Since the relation is well-founded, the looping and recursion must eventually halt. A typical total correctness property asserts that the program halts *and* its output satisfies some relation with its input. The ranking function method applies to the first conjunct (the program halts); the inductive assertion method applies to the second (its output satisfies some relation with its input). Section 5.3 presents the ranking function method.

We explain all concepts in this and the next chapter with a set of example programs that manipulate arrays. We chose these programs for several reasons. First, they should be familiar to most readers; the reader can thus focus on the verification methodology rather than on understanding new complex programs. Second, our correctness proofs rely heavily on the decision procedures that are discussed in Part II of this book. Finally, they are small but dense, allowing us to exhibit common techniques for proving interesting facts about programs. The reader should keep in mind, however, that the methods of this chapter underlie software and hardware analyses that are applied in practice.

The reader may find the contents of this chapter to be rather technical. Indeed, the transformation of an annotated program into a set of verification conditions is a purely mechanical task. Fortunately, a **verifying compiler** does this work in practice: it parses an annotated program, checking the syntax and semantics as usual, and generates a set of verification conditions. But just as learning how compilers work is important for understanding programming languages, learning how verifying compilers work is important for understanding verification and programming languages with annotations. Moreover, applying the steps of generating verification conditions provides practice in manipulating and understanding FOL formulae, a useful skill. Chapter 6 discusses strategies for writing annotations, a task that cannot be fully automated.

5.1 pi: A Simple Imperative Language

This section introduces the programming language **pi**, an imperative language with facilities for annotations. To allow us to focus on the fundamentals of program verification, **pi** lacks complicating features of typical imperative lan-

```

@pre T
@post T
bool LinearSearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
    for @ T
        (int  $i := \ell$ ;  $i \leq u$ ;  $i := i + 1$ ) {
            if ( $a[i] = e$ ) return true;
        }
    return false;
}

```

Fig. 5.1. LinearSearch

guages: its data types do not include pointer or reference types; and it does not allow global variables, although it does have global constants (see Exercise 6.5). After reading this chapter and Chapter 12, the interested reader should consult the wide literature on program analysis to learn how the techniques of these chapters extend to reasoning about standard programming languages.

5.1.1 The Language

Because **pi** is superficially a C-like language with restrictions, we present the essential features of **pi** through examples.

Example 5.1. Figure 5.1 lists the function **LinearSearch**, which searches the range $[\ell, u]$ of an array a of integers for a value e . It returns **true** iff the given array contains the value between the lower bound ℓ and upper bound u . It behaves correctly only if $0 \leq \ell$ and $u < |a|$; otherwise, the array a is accessed outside of its domain $[0, |a| - 1]$. $|a|$ denotes the length of array a .

Observe that most of the syntax is similar to C. For example, the **for** loop sets i to be ℓ initially and then executes the body of the loop and increments i by 1 as long as $i \leq u$. Also, an integer array has type **int** $[]$, which is constructed from base type **int**. One syntactic difference occurs in assignment, which is written $:=$ to distinguish it from the equality predicate $=$. We use $=$ as the equality predicate, rather than $==$, to correspond to the standard equality predicate of FOL. Finally, unlike C, **pi** has type **bool** and constants **true** and **false**.

Notice the lines beginning with **@**. They are program annotations, which we discuss in detail in the next section.

In **LinearSearch**, a , ℓ , u , and e are the **formal parameters** (also, **parameters**) of the function. If **LinearSearch** is called as **LinearSearch**($b, 0, |b| - 1, v$), then b , 0, $|b| - 1$, and v are the **arguments**. ■

Example 5.2. Figure 5.2 lists the recursive function **BinarySearch**, which searches a range $[\ell, u]$ of a sorted (weakly increasing: $a[i] \leq a[j]$ if $i \leq j$) array a of integers for a value e . Like **LinearSearch**, it returns **true** iff the

```

@pre T
@post T
bool BinarySearch(int[] a, int ℓ, int u, int e) {
    if (ℓ > u) return false;
    else {
        int m := (ℓ + u) div 2;
        if (a[m] = e) return true;
        else if (a[m] < e) return BinarySearch(a, m + 1, u, e);
        else return BinarySearch(a, ℓ, m - 1, e);
    }
}

```

Fig. 5.2. BinarySearch

given array contains the value in the range $[\ell, u]$. It behaves correctly only if $0 \leq \ell$ and $u < |a|$.

One level of recursion operates as follows. If the lower bound ℓ of the range is greater than the upper bound u , then the (empty) subarray cannot contain e , so it returns **false**. Otherwise, it examines the middle element $a[m]$ of the subarray: if it is e , then the subarray clearly contains e ; otherwise, it recurses on the left half if $a[m] < e$ and on the right half if $a[m] > e$.

pi syntactically distinguishes between integer division and real division: for **int** variables a and b , write $a \text{ div } b$ instead of a/b . Integer division is defined as follows:

$$a \text{ div } b \stackrel{\text{def}}{=} \left\lfloor \frac{a}{b} \right\rfloor.$$

That is, $a \text{ div } b$ is equal to the greatest integer less than or equal to $\frac{a}{b}$ (the *floor* of $\frac{a}{b}$). ■

Example 5.3. Figure 5.3 lists the function **BubbleSort**, which sorts an integer array. It works by “bubbling” the largest element of the left unsorted region of the array toward the sorted region on the right; this element then becomes the left element of the sorted region, enlarging the region by one cell. In Figure 5.4, for example, the first line shows an array in which the rightmost boxed cells comprise the sorted region and the other cells comprise the unsorted region. In the final line, the sorted region has been expanded by one cell.

Figure 5.4 lists a portion of a sample execution trace. The right two cells (5, 6) of the array have already been sorted. In the trace, the inner loop moves the largest element 4 of the unsorted region to the right to join the sorted region, which is indicated by the dotted rectangle. In the first two steps, $a[j] \leq a[j+1]$ ($2 \leq 3$ and $3 \leq 4$), so the values of cell j and $j+1$ are not swapped in either case. In the subsequent two steps, $a[j] > a[j+1]$ ($4 > 1$ and $4 > 2$), causing a swap at each step. In the fifth step, the inner loop’s guard $i < j$ no longer holds, so the inner loop exits and the outer

```

@pre  $\top$ 
@post  $\top$ 
int[] BubbleSort(int[]  $a_0$ ) {
  int[]  $a := a_0$ ;
  for @  $\top$ 
    (int  $i := |a| - 1$ ;  $i > 0$ ;  $i := i - 1$ ) {
      for @  $\top$ 
        (int  $j := 0$ ;  $j < i$ ;  $j := j + 1$ ) {
          if ( $a[j] > a[j + 1]$ ) {
            int  $t := a[j]$ ;
             $a[j] := a[j + 1]$ ;
             $a[j + 1] := t$ ;
          }
        }
      }
    }
  return  $a$ ;
}

```

Fig. 5.3. BubbleSort

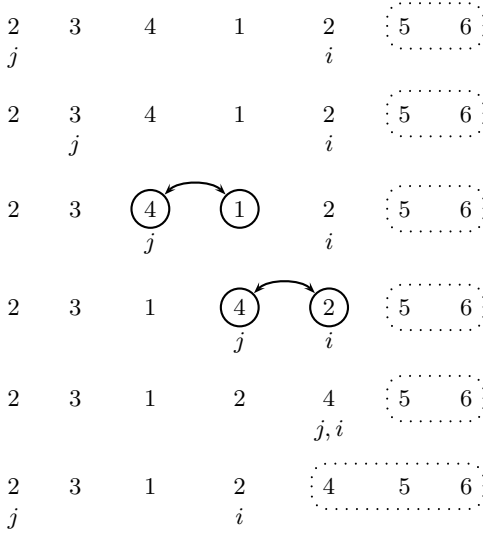


Fig. 5.4. Sample execution of BubbleSort

loop decrements i by 1. The sorted region has been expanded by one cell, as indicated by the final dotted rectangle. The last step shows the beginning of the next round of the inner loop.

Because pi does not have pointer or reference types, all data are passed by value, including arrays and structures. If BubbleSort were missing the **return**

```
typedef struct qs {
    int pivot;
    int[] array;
} qs;
```

Fig. 5.5. Structure `qs`

statement, then calling it would not have any discernible effect on the calling context. Additionally, `pi` does not allow updates to parameters, so `BubbleSort` assigns a_0 to a fresh variable a in the first line. This artificial requirement makes reasoning about functions easier: in annotations (see Section 5.1.2) throughout the function, one can always reference the input. ■

In this book, our example programs manipulate arrays rather than recursive data structures. The reason is that we can express more interesting properties about arrays in the fragment of the theory of arrays studied in Chapter 11 than we can about lists in the fragment of the theory of recursive data structures studied in Chapter 9. This bias is a reflection of the structure and content of this book, not of what is theoretically possible.

However, we sometimes use records, a basic recursive data type, to allow a function to return multiple values. The following example illustrates such a record type, which is used in the program `QuickSort` (see Section 6.2).

Example 5.4. The structure `qs` of Figure 5.5 is a record with two fields: the *pivot* field of type `int` and the *array* field of type `array`. If x is a variable of type `qs`, then $x.pivot$ returns the value in its *pivot* field; also, $x.array[i] := v$ assigns v to position i of x 's *array* field. ■

5.1.2 Program Annotations

The most important feature of `pi` is the capacity for complex function annotations. An **annotation** is a FOL formula F whose free variables include only the program variables of the function in which the annotation occurs. An annotation F at location L asserts that F is true whenever program control reaches L . We discuss several forms of annotations in this section.

Function Specifications

The **function specification** of a function is a pair of annotations. The **function precondition** is a formula F whose free variables include only the formal parameters. It specifies what should be true upon entering the function — or, in other words, under what inputs the function is expected to work. The **function postcondition** is a formula G whose free variables include only the formal parameters and the special variable rv representing the return value of the function. The postcondition relates the function's output (the return value rv) to its input (the parameters).

```

@pre  $0 \leq \ell \wedge u < |a|$ 
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 
bool LinearSearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
    for @  $\top$ 
        (int  $i := \ell$ ;  $i \leq u$ ;  $i := i + 1$ ) {
            if ( $a[i] = e$ ) return true;
        }
    return false;
}

```

Fig. 5.6. LinearSearch with function specification

```

@pre  $\top$ 
@post  $rv \leftrightarrow \exists i. 0 \leq \ell \leq i \leq u < |a| \wedge a[i] = e$ 
bool LinearSearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
    if ( $\ell < 0 \vee u \geq |a|$ ) return false;
    for @  $\top$ 
        (int  $i := \ell$ ;  $i \leq u$ ;  $i := i + 1$ ) {
            if ( $a[i] = e$ ) return true;
        }
    return false;
}

```

Fig. 5.7. Robust LinearSearch with function specification

Example 5.5. In Example 5.1, we informally specified the behavior of LinearSearch as follows: LinearSearch returns **true** iff the array a contains the value e in the range $[\ell, u]$. It behaves correctly only when $\ell \geq 0$ and $u < |a|$.

Function specifications formalize such statements. Figure 5.6 presents LinearSearch with its specification. The precondition asserts that the lower bound ℓ should be at least 0 and that the upper bound u should be less than the length $|a|$ of the array a . The postcondition asserts that the return value rv is **true** iff $a[i] = e$ for some index $i \in [\ell, u]$ of a . ■

Example 5.6. A nontrivial precondition (a formula other than $\top$) is not always acceptable, especially if a function is public to a module. Figure 5.7 lists a more robust version of linear search. The formula

$$0 \leq \ell \leq i \leq u < |a|$$

abbreviates

$$0 \leq \ell \wedge \ell \leq i \wedge i \leq u \wedge u < |a|.$$

A nontrivial precondition is sometimes acceptable for a function that is private to a module. The verification method of this chapter checks that every instance of a call to such a function obeys the precondition. ■

```

@pre  $0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u)$ 
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 
bool BinarySearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
  if ( $\ell > u$ ) return false;
  else {
    int  $m := (\ell + u) \text{ div } 2$ ;
    if ( $a[m] = e$ ) return true;
    else if ( $a[m] < e$ ) return BinarySearch( $a, m + 1, u, e$ );
    else return BinarySearch( $a, \ell, m - 1, e$ );
  }
}

```

Fig. 5.8. BinarySearch with function specification

```

@pre  $\top$ 
@post sorted( $rv, 0, |rv| - 1$ )
int[] BubbleSort(int[]  $a_0$ ) {
  int[]  $a := a_0$ ;
  for @  $\top$ 
    (int  $i := |a| - 1; i > 0; i := i - 1$ ) {
      for @  $\top$ 
        (int  $j := 0; j < i; j := j + 1$ ) {
          if ( $a[j] > a[j + 1]$ ) {
            int  $t := a[j]$ ;
             $a[j] := a[j + 1]$ ;
             $a[j + 1] := t$ ;
          }
        }
      }
  }
  return  $a$ ;
}

```

Fig. 5.9. BubbleSort with function specification

Example 5.7. Figure 5.8 lists BinarySearch with its specification. As expected, its postcondition is identical to the postcondition of LinearSearch. However, its precondition also states that the array a is sorted.

The **sorted** predicate is defined in the combined theory of integers and arrays, $T_{\mathbb{Z}} \cup T_A$:

$$\text{sorted}(a, \ell, u) \Leftrightarrow \forall i, j. \ell \leq i \leq j \leq u \rightarrow a[i] \leq a[j] .$$

■

Example 5.8. Figure 5.9 lists BubbleSort with its specification. Given any array, the returned array is sorted. Of course, other properties are desirable

and could be specified as well. For example, the returned array rv should be a permutation of the original array a_0 (see Exercise 6.5). ■

Section 5.2 presents a method for proving that a function satisfies its partial correctness specification: if the function precondition is satisfied and the function halts, then the function postcondition holds upon return. Section 5.3 discusses a method for proving that, additionally, the function always halts.

Loop Invariants

Each **for** loop and **while** loop has an attendant annotation called the **loop invariant**. A **while** loop

```
while
  @  $F$ 
  ( $\langle condition \rangle$ ) {
     $\langle body \rangle$ 
  }
```

says to apply the $\langle body \rangle$ as long as $\langle condition \rangle$ holds. The assertion F must hold at the beginning of every iteration. It is evaluated before the $\langle condition \rangle$ is evaluated, so it must hold even on the final iteration when $\langle condition \rangle$ is **false**. Therefore, on entering the $\langle body \rangle$ of the loop,

$$F \wedge \langle condition \rangle$$

must hold, and on exiting the loop,

$$F \wedge \neg \langle condition \rangle$$

must hold.

To consider a **for** loop, translate the loop

```
for
  @  $F$ 
  ( $\langle initialize \rangle$ ;  $\langle condition \rangle$ ;  $\langle increment \rangle$ ) {
     $\langle body \rangle$ 
  }
```

into the equivalent loop

```
 $\langle initialize \rangle$ ;
while
  @  $F$ 
  ( $\langle condition \rangle$ ) {
     $\langle body \rangle$ 
     $\langle increment \rangle$ 
  }
```

F must hold after the $\langle initialize \rangle$ statement has been evaluated and, on each iteration, before the $\langle condition \rangle$ is evaluated.

```

@pre  $0 \leq \ell \wedge u < |a|$ 
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 
bool LinearSearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
  for
    @L :  $\ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$ 
    (int  $i := \ell$ ;  $i \leq u$ ;  $i := i + 1$ ) {
      if ( $a[i] = e$ ) return true;
    }
  return false;
}

```

Fig. 5.10. LinearSearch with loop invariant

Example 5.9. Figure 5.10 lists LinearSearch with a nontrivial loop invariant at L . It asserts that whenever control reaches L , the loop index is at least ℓ and that $a[j] \neq e$ for previously examined indices j . ■

Section 5.2 shows that loop invariants are crucial for constructing an inductive argument that a function obeys its specification.

Assertions

In **pi**, one can add an annotation anywhere. When an annotation is not a function precondition, function postcondition, or loop invariant, we call it an **assertion**. Assertions allow programmers to provide a formal comment. For example, if at the statement

$$i := i + k;$$

the programmer thinks that k is positive, then the programmer can add an assertion stating that supposition:

$$\begin{aligned} &@ \ k > 0; \\ &i := i + k; \end{aligned}$$

Later, the programmer's hypothesis about k is verified with formal verification at compile time or with dynamic assertion tests at runtime.

Runtime assertions are a special class of assertions. In most programming languages, **runtime errors** include division by 0, modulo by 0, and dereference of **null**. In particular, division by 0 and modulo by 0 cause hardware exceptions, while only some languages, such as Java, catch a dereference of **null**. In **pi**, runtime errors include division by 0, modulo by 0, and accessing an array out of bounds. The **pi** compiler generates runtime assertions to catch runtime errors.

Example 5.10. Figure 5.11 lists LinearSearch with runtime assertions. The array read $a[i]$ is protected by the assertion that i is a legal index of a . ■

```

@pre T
@post T
bool LinearSearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
    for @ T
        (int  $i := \ell$ ;  $i \leq u$ ;  $i := i + 1$ ) {
            @  $0 \leq i < |a|$ ;
            if ( $a[i] = e$ ) return true;
        }
    return false;
}

```

Fig. 5.11. LinearSearch with runtime assertions

```

@pre T
@post T
bool BinarySearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
    if ( $\ell > u$ ) return false;
    else {
        @  $2 \neq 0$ ;
        int  $m := (\ell + u) \text{ div } 2$ ;
        @  $0 \leq m < |a|$ ;
        if ( $a[m] = e$ ) return true;
        else {
            @  $0 \leq m < |a|$ ;
            if ( $a[m] < e$ ) return BinarySearch( $a, m + 1, u, e$ );
            else return BinarySearch( $a, \ell, m - 1, e$ );
        }
    }
}

```

Fig. 5.12. BinarySearch with runtime assertions

Example 5.11. Figure 5.12 lists BinarySearch with runtime assertions. The first assertion protects the division: it asserts that $2 \neq 0$, which clearly holds. The next two assertions protect the array reads. ■

Example 5.12. Figure 5.13 lists BubbleSort with compiler-generated runtime assertions. All assertions protect array accesses. The first two runtime assertions are sufficient to protect all array accesses. Figure 5.14 lists a concise version. ■

5.2 Partial Correctness

Having specified and implemented each function of a program, we would like to prove that the functions obey their specifications (from another perspective,

```

@pre T
@post T
int[] BubbleSort(int[] a0) {
  int[] a := a0;
  for @ T
    (int i := |a| - 1; i > 0; i := i - 1) {
      for @ T
        (int j := 0; j < i; j := j + 1) {
          @ 0 ≤ j < |a|;
          @ 0 ≤ j + 1 < |a|;
          if (a[j] > a[j + 1]) {
            @ 0 ≤ j < |a|;
            int t := a[j];
            @ 0 ≤ j < |a|;
            @ 0 ≤ j + 1 < |a|;
            a[j] := a[j + 1];
            @ 0 ≤ j + 1 < |a|;
            a[j + 1] := t;
          }
        }
      }
    }
  return a;
}

```

Fig. 5.13. BubbleSort with runtime assertions

that their specifications reflect their actual behavior). A function is **partially correct** if when the function's precondition is satisfied on entry, its postcondition is satisfied when the function returns (if it ever does). In general, functions may not halt: a loop's guard could be incorrect, or a recursive function could fail to handle a particular base case. Section 5.3 discusses how to prove that a function always halts.

We present the **inductive assertion method** for proving that a program is partially correct. The method reduces each function and its annotations to a finite set of **verification conditions** (VCs), which are FOL formulae. If all of a function's VCs are valid, then the function obeys its specification. The reduction occurs in two stages: first, each function of the annotated program is broken down into a finite set of **basic paths** (Sections 5.2.1 and 5.2.2); second, each basic path generates a verification condition (Section 5.2.4). Sections 5.2.3 and 5.2.5 provide a more abstract view of the inductive assertion method.

Loops complicate proofs of partial correctness because they create an unbounded number of paths from function entry to exit. Recursive functions similarly complicate proofs. A **path** is a sequence of program statements. For loops, loop invariants cut the paths into a finite set of basic paths, while for recursive functions, the function specification of the recursive function cuts the paths. In this section, we assume that we are given function specifications

```

@pre  $\top$ 
@post  $\top$ 
int[] BubbleSort(int[]  $a_0$ ) {
  int[]  $a := a_0$ ;
  for @  $\top$ 
    (int  $i := |a| - 1$ ;  $i > 0$ ;  $i := i - 1$ ) {
      for @  $\top$ 
        (int  $j := 0$ ;  $j < i$ ;  $j := j + 1$ ) {
          @  $0 \leq j < |a| \wedge 0 \leq j + 1 < |a|$ ;
          if ( $a[j] > a[j + 1]$ ) {
            int  $t := a[j]$ ;
             $a[j] := a[j + 1]$ ;
             $a[j + 1] := t$ ;
          }
        }
      }
    }
  return  $a$ ;
}

```

Fig. 5.14. BubbleSort with compressed runtime assertions

```

@pre  $0 \leq \ell \wedge u < |a|$ 
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 
bool LinearSearch(int[]  $a$ , int  $\ell$ , int  $u$ , int  $e$ ) {
  for
    @L:  $\ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$ 
    (int  $i := \ell$ ;  $i \leq u$ ;  $i := i + 1$ ) {
      if ( $a[i] = e$ ) return true;
    }
  return false;
}

```

Fig. 5.15. LinearSearch with loop invariants

and loop invariants and concentrate on the task of generating the corresponding verification conditions. In practice, this task is performed by a verifying compiler. Chapter 6 discusses strategies for constructing specifications and loop invariants.

5.2.1 Basic Paths: Loops

A **basic path** is a sequence of instructions that begins at the function precondition or a loop invariant and ends at a loop invariant, an assertion, or the function postcondition. Moreover, a loop invariant can only occur at the beginning or the ending of a basic path. Thus, basic paths do not cross loops. The following examples illustrate the characteristics of basic paths.

Example 5.13. Figure 5.15 lists an annotated version of `LinearSearch`. Its basic paths are the following. The first basic path starts at the function precondition, enters the `for` loop via the initialization statement, and ends at the loop invariant L :

(1)

```
@pre  $0 \leq \ell \wedge u < |a|$ 
 $i := \ell;$ 
@L :  $\ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$ 
```

The second basic path begins at the loop invariant at L , passes the loop guard $i \leq u$, passes the guard $a[i] = e$ of the `if` statement, executes the `return` (of `true`), and ends at the postcondition:

(2)

```
@L :  $\ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$ 
assume  $i \leq u;$ 
assume  $a[i] = e;$ 
 $rv := \text{true};$ 
@post  $rv \leftrightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e$ 
```

This path exhibits two new aspects of basic paths. First, `return` statements become assignments to the special variable rv representing the return value.

Second, guards arising in program statements (in `for` loop guards, `while` loop guards, or `if` statements) become **assume statements** in basic paths. An assume statement **assume** c in a basic path means that the remainder of the basic path is executed only if the condition c holds at **assume** c . Each guard with condition c results in two assumptions: the guard holds (c) or it does not hold ($\neg c$). Therefore, each guard produces two paths with the same prefix up to the guard. They diverge on the assumption: one basic path has the statement **assume** c , and the other has the statement **assume** $\neg c$. These assumptions and the control structure of the program determine the construction of the remainder of the basic paths.

For example, the third path has the same prefix as (2) but makes the opposite assumption at the `if` statement guard: it assumes $a[i] \neq e$ rather than $a[i] = e$. Therefore, this path loops back around to the loop invariant:

(3)

```
@L :  $\ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$ 
assume  $i \leq u;$ 
assume  $a[i] \neq e;$ 
 $i := i + 1;$ 
@L :  $\ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$ 
```

The final basic path has the same prefix as (2) and (3) but makes the opposite assumption at the `for` loop guard: it assumes $i > u$ rather than $i \leq u$. Therefore, this path exits the loop and returns **false**:

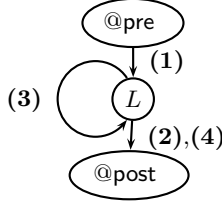


Fig. 5.16. Visualization of basic paths of LinearSearch

(4)

$@L : \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e)$
assume $i > u$;
 $rv := \text{false}$;
 $@\text{post } rv \leftrightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e$

To avoid forgetting a basic path, we list paths in a depth-first order. When a guard is encountered, assume that it holds and generate the resulting paths; then assume that it does not hold and generate the resulting paths. In this example, the loop guard $i \leq u$ in (2) is first encountered; (2) and (3) follow from the assumption that the loop guard holds, while (4) follows from the assumption that it does not hold. The **if** statement guard $a[i] = e$ is next encountered in (2); (2) follows from the assumption that it holds, while (3) follows from the assumption that it does not hold. Figure 5.16 visualizes these basic paths. ■

Example 5.14. Figure 5.17 lists BubbleSort with loop invariants. The outer loop invariant at L_1 asserts that

- i is in the range $[-1, |a| - 1]$ (if $|a| = 0$, then i is initially -1);
- a is sorted in the range $[i, |a| - 1]$;
- and a is **partitioned** such that each element in the range $[0, i]$ is at most (less than or equal to) each element in the range $[i + 1, |a| - 1]$.

Its inner loop invariant at L_2 asserts that

- i is in the range $[1, |a| - 1]$, and j is in the range $[0, i]$;
- a is sorted in the range $[i, |a| - 1]$ as in the outer loop;
- a is partitioned as in the outer loop;
- and a is also partitioned such that each element in the range $[0, j - 1]$ is at most $a[j]$.

The **partitioned** predicate is defined in the theory $T_Z \cup T_A$:

$\text{partitioned}(a, \ell_1, u_1, \ell_2, u_2)$
 $\Leftrightarrow \forall i, j. \ell_1 \leq i \leq u_1 < \ell_2 \leq j \leq u_2 \rightarrow a[i] \leq a[j].$

```

@pre  $\top$ 
@post sorted( $rv, 0, |rv| - 1$ )
int[] BubbleSort(int[]  $a_0$ ) {
  int[]  $a := a_0$ ;
  for
    @ $L_1 : \left[ \begin{array}{l} -1 \leq i < |a| \\ \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \\ \wedge \text{sorted}(a, i, |a| - 1) \end{array} \right]$ 
    (int  $i := |a| - 1$ ;  $i > 0$ ;  $i := i - 1$ ) {
    for
      @ $L_2 : \left[ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \\ \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \\ \wedge \text{partitioned}(a, 0, j - 1, j, j) \\ \wedge \text{sorted}(a, i, |a| - 1) \end{array} \right]$ 
      (int  $j := 0$ ;  $j < i$ ;  $j := j + 1$ ) {
      if ( $a[j] > a[j + 1]$ ) {
        int  $t := a[j]$ ;
         $a[j] := a[j + 1]$ ;
         $a[j + 1] := t$ ;
      }
    }
  }
  return  $a$ ;
}

```

Fig. 5.17. BubbleSort with loop invariants

Performing a depth-first exploration, the first basic path starts at the pre-condition and ends at the outer loop invariant at L_1 :

(1)

```

@pre  $\top$ ;
 $a := a_0$ ;
 $i := |a| - 1$ ;
@ $L_1 : -1 \leq i < |a| \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \wedge \text{sorted}(a, i, |a| - 1)$ 

```

The second basic path starts at L_1 and ends at the inner loop invariant at L_2 (recall that the annotation is checked after the loop initialization $j := 0$):

(2)

```

@ $L_1 : -1 \leq i < |a| \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \wedge \text{sorted}(a, i, |a| - 1)$ 
assume  $i > 0$ ;
 $j := 0$ ;
@ $L_2 : \left[ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \\ \wedge \text{partitioned}(a, 0, j - 1, j, j) \wedge \text{sorted}(a, i, |a| - 1) \end{array} \right]$ 

```

The third and fourth basic paths follow the inner loop, each handling one assumption on the guard $a[j] > a[j + 1]$ of the **if** statement:

(3)

```

@L2 : [ 1 ≤ i < |a| ∧ 0 ≤ j ≤ i ∧ partitioned(a, 0, i, i + 1, |a| - 1)
        ∧ partitioned(a, 0, j - 1, j, j) ∧ sorted(a, i, |a| - 1) ]
assume j < i;
assume a[j] > a[j + 1];
t := a[j];
a[j] := a[j + 1];
a[j + 1] := t;
j := j + 1;
@L2 : [ 1 ≤ i < |a| ∧ 0 ≤ j ≤ i ∧ partitioned(a, 0, i, i + 1, |a| - 1)
        ∧ partitioned(a, 0, j - 1, j, j) ∧ sorted(a, i, |a| - 1) ]

```

(4)

```

@L2 : [ 1 ≤ i < |a| ∧ 0 ≤ j ≤ i ∧ partitioned(a, 0, i, i + 1, |a| - 1)
        ∧ partitioned(a, 0, j - 1, j, j) ∧ sorted(a, i, |a| - 1) ]
assume j < i;
assume a[j] ≤ a[j + 1];
j := j + 1;
@L2 : [ 1 ≤ i < |a| ∧ 0 ≤ j ≤ i ∧ partitioned(a, 0, i, i + 1, |a| - 1)
        ∧ partitioned(a, 0, j - 1, j, j) ∧ sorted(a, i, |a| - 1) ]

```

The fifth basic path starts at L_2 , exits the inner loop, and decrements i on its way to L_1 :

(5)

```

@L2 : [ 1 ≤ i < |a| ∧ 0 ≤ j ≤ i ∧ partitioned(a, 0, i, i + 1, |a| - 1)
        ∧ partitioned(a, 0, j - 1, j, j) ∧ sorted(a, i, |a| - 1) ]
assume j ≥ i;
i := i - 1;
@L1 : -1 ≤ i < |a| ∧ partitioned(a, 0, i, i + 1, |a| - 1) ∧ sorted(a, i, |a| - 1)

```

The final basic path starts at L_1 , exits the outer loop, and then exits the function, returning the (presumably sorted) array a :

(6)

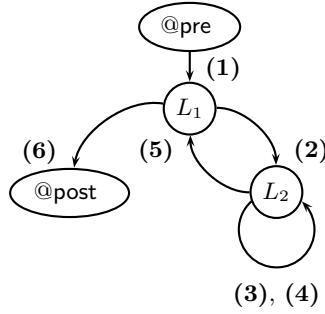
```

@L1 : -1 ≤ i < |a| ∧ partitioned(a, 0, i, i + 1, |a| - 1) ∧ sorted(a, i, |a| - 1)
assume i ≤ 0;
rv := a;
@post sorted(rv, 0, |rv| - 1)

```

Figure 5.18 visualizes these basic paths. ■

Example 5.15. Figure 5.19 lists BubbleSort with runtime assertions at L_3 and a different set of loop invariants at L_1 and L_2 relevant for proving the runtime assertions. Six basic paths correspond in structure to those of Figure 5.18, although their content varies based on the new precondition, postcondi-

**Fig. 5.18.** Visualization of basic paths of BubbleSort

```

@pre ⊤
@post ⊤
int[] BubbleSort(int[] a0) {
  int[] a := a0;
  for
    @L1 : -1 ≤ i < |a|
    (int i := |a| - 1; i > 0; i := i - 1) {
    for
      @L2 : 0 < i < |a| ∧ 0 ≤ j ≤ i
      (int j := 0; j < i; j := j + 1) {
      @L3 : 0 ≤ j < |a| ∧ 0 ≤ j + 1 < |a|;
      if (a[j] > a[j + 1]) {
        int t := a[j];
        a[j] := a[j + 1];
        a[j + 1] := t;
      }
    }
  }
  return a;
}

```

Fig. 5.19. BubbleSort with runtime assertions

tion, and loop invariants. These basic paths ignore the runtime assertion at L_3 . Then one additional basic path ends at the runtime assertion:

```

(7)
@L2 : 0 < i < |a| ∧ 0 ≤ j ≤ i
assume j < i;
@L3 : 0 ≤ j < |a| ∧ 0 ≤ j + 1 < |a|

```

■

```

@pre  $0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u)$ 
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 
bool BinarySearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
  if ( $\ell > u$ ) return false;
  else {
    int  $m := (\ell + u) \text{ div } 2$ ;
    if ( $a[m] = e$ ) return true;
    else if ( $a[m] < e$ ) {
      @ $R_1$ :  $0 \leq m + 1 \wedge u < |a| \wedge \text{sorted}(a, m + 1, u)$ ;
      return BinarySearch( $a, m + 1, u, e$ );
    } else {
      @ $R_2$ :  $0 \leq \ell \wedge m - 1 < |a| \wedge \text{sorted}(a, \ell, m - 1)$ ;
      return BinarySearch( $a, \ell, m - 1, e$ );
    }
  }
}

```

Fig. 5.20. BinarySearch with function call assertions

5.2.2 Basic Paths: Function Calls

Like loops, recursive functions create an unbounded number of paths within programs. But just as loop invariants cut loops to produce a finite number of basic paths, function specifications cut function calls.

Recall that the function postcondition is a relation between the return value rv and the formal parameters. A function's postcondition summarizes the effects of calling it. We use these summaries to replace function calls in basic paths.

Remark 5.16. The postconditions of the functions of a program need only include information that is relevant for proving the given specification, so the summaries may be incomplete. Ignoring irrelevant aspects of functions reduces the size of annotations. Chapter 6 discusses techniques for developing function specifications.

The replacement of function calls by function summaries makes the listing of basic paths (and the resulting analysis described in Section 5.2.4) local to functions. Basic paths do not span multiple functions. However, recall that the function postcondition is guaranteed to hold on return only when the function precondition is satisfied on entry. To ensure that the precondition is satisfied, each instance of a function call generates an extra basic path in which the called function's precondition is asserted. An example clarifies this discussion.

Example 5.17. Figure 5.8 lists BinarySearch with its function specification. BinarySearch contains two (recursive) function calls. In Figure 5.20, each function call is protected by a **function call assertion** at R_1 and R_2 . Each assertion is constructed by applying a substitution to BinarySearch's precondition

$$F[a, \ell, u, e] : 0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u) .$$

The first function call is $\text{BinarySearch}(a, m + 1, u, e)$, so the function call assertion at R_1 is $F\sigma_1$, where

$$\sigma_1 : \{a \mapsto a, \ell \mapsto m + 1, u \mapsto u, e \mapsto e\} .$$

The notation of Section 1.5 allows us to write $F[a, m + 1, u, e]$.

The second function call is $\text{BinarySearch}(a, \ell, m - 1, e)$, so the function call assertion at R_2 is $F[a, \ell, m - 1, e]$. These assertions are treated in the same way as other assertions, such as runtime assertions. So far, we have the following basic paths:

(1)

```
@pre 0 ≤ ℓ ∧ u < |a| ∧ sorted(a, ℓ, u)
assume ℓ > u;
rv := false;
@post rv ↔ ∃i. ℓ ≤ i ≤ u ∧ a[i] = e
```

(2)

```
@pre 0 ≤ ℓ ∧ u < |a| ∧ sorted(a, ℓ, u)
assume ℓ ≤ u;
m := (ℓ + u) div 2;
assume a[m] = e;
rv := true;
@post rv ↔ ∃i. ℓ ≤ i ≤ u ∧ a[i] = e
```

(3)

```
@pre 0 ≤ ℓ ∧ u < |a| ∧ sorted(a, ℓ, u)
assume ℓ ≤ u;
m := (ℓ + u) div 2;
assume a[m] ≠ e;
assume a[m] < e;
@R1 : 0 ≤ m + 1 ∧ u < |a| ∧ sorted(a, m + 1, u)
```

(5)

```
@pre 0 ≤ ℓ ∧ u < |a| ∧ sorted(a, ℓ, u)
assume ℓ ≤ u;
m := (ℓ + u) div 2;
assume a[m] ≠ e;
assume a[m] ≥ e;
@R2 : 0 ≤ ℓ ∧ m - 1 < |a| ∧ sorted(a, ℓ, m - 1)
```

Because BinarySearch lacks loops, each basic path starts at the function precondition.

It remains to consider paths (4) and (6), which pass through the recursive function calls and end at the postcondition. Paths (3) and (5) end in the function call assertions at R_1 and R_2 protecting these function calls. Since they assert that the called `BinarySearch`'s precondition holds, we can assume that the returned values obey the postcondition of `BinarySearch` in each of the calling contexts. Therefore, we can use the function postcondition as a summary of the function call:

(4)

```

@pre  $0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u)$ 
assume  $\ell \leq u$ ;
 $m := (\ell + u) \text{ div } 2$ ;
assume  $a[m] \neq e$ ;
assume  $a[m] < e$ ;
assume  $v_1 \leftrightarrow \exists i. m + 1 \leq i \leq u \wedge a[i] = e$ ;
 $rv := v_1$ ;
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 

```

The lines

```

assume  $v_1 \leftrightarrow \exists i. m + 1 \leq i \leq u \wedge a[i] = e$ ;
 $rv := v_1$ ;

```

arise as follows. First, translate the `return` statement

```
return BinarySearch( $a, m + 1, u, e$ );
```

into an assignment to rv , as usual:

```
 $rv := \text{BinarySearch}(a, m + 1, u, e);$ 
```

Next, given that the precondition holds (from path (3)), assume that the postcondition holds. Therefore, summarize the function call with a relation based on `BinarySearch`'s postcondition,

$$G[a, \ell, u, e, rv] : rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e .$$

Specifically, the relation is $G[a, m + 1, u, e, v_1]$, where v_1 is a fresh variable that captures the return value. In the basic path, assume this relation; then use the return value v_1 in the assignment:

```

assume  $G[a, m + 1, u, e, v_1]$ ;
 $rv := v_1$ ;

```

These are the penultimate lines of (4). Hence, (4) replaces the function call `BinarySearch( $a, m + 1, u, e$ )` with a summary based on the function postcondition. Now reasoning about the basic path does not require reasoning about all of `BinarySearch` at once.

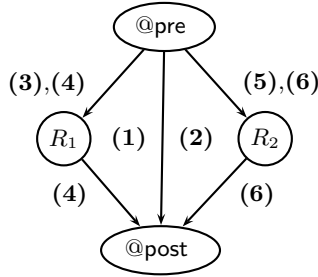


Fig. 5.21. Visualization of basic paths of BinarySearch

Construct the final basic path for the function call $\text{BinarySearch}(a, \ell, m - 1, e)$ similarly:

(6)

```

@pre  $0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u)$ 
assume  $\ell \leq u$ ;
 $m := (\ell + u) \text{ div } 2$ ;
assume  $a[m] \neq e$ ;
assume  $a[m] \geq e$ ;
assume  $v_2 \leftrightarrow \exists i. \ell \leq i \leq m - 1 \wedge a[i] = e$ ;
 $rv := v_2$ ;
@post  $rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e$ 

```

Again, v_2 is a fresh variable.

Figure 5.21 visualizes these basic paths. Paths (4) and (6) are shown to pass through locations R_1 and R_2 , respectively. ■

For the general case, consider function f with prototype

```

@pre  $F[p_1, \dots, p_n]$ 
@post  $G[p_1, \dots, p_n, rv]$ 
type0  $f(\text{type}_1 p_1, \dots, \text{type}_n p_n)$ 

```

Suppose that f is called in context

$$w := f(e_1, \dots, e_n);$$

where $e_1, \dots, e_n$ are expressions. Then augment the calling context with the function call assertion:

```

@  $F[e_1, \dots, e_n]$ ;
 $w := f(e_1, \dots, e_n);$ 

```

Treat this new assertion the same as any assertion: it results in at least one basic path ending in

```

...
@  $F[e_1, \dots, e_n]$ 

```

Finally, in basic paths that pass through the function call, replace the function call by an assumption and assignment constructed from the postcondition, where v is a fresh variable:

```
...
assume  $G[e_1, \dots, e_n, v]$ ;
 $w := v$ ;
...
```

Note that rv need not have type `bool` as in `BinarySearch`. For example, for a function with prototype

```
@pre  $\top$ 
@post  $rv \geq x$ 
int  $g(\text{int } x)$ 
```

the statement

```
 $w := g(n + 1)$ ;
```

is summarized in basic paths as follows:

```
assume  $v \geq n + 1$ ;
 $w := v$ ;
```

5.2.3 Program States

Before presenting the final step in proving partial correctness, we formalize program state. A program **state** s is an assignment of values of the proper type to all variables. The program variables include a distinguished variable pc , the **program counter**. It holds the current location of control.

Example 5.18. The state

$$s : \{pc \mapsto L_1, a \mapsto [2; 0; 1], i \mapsto 2, j \mapsto 0, t \mapsto 2, rv \mapsto []\}$$

is a state of `BubbleSort` in which control resides at L_1 . ■

A state can be extended to a logical interpretation. Suppose that T is the theory that captures the functions ($+$, $-$, *etc.*) and predicates ($=$, $<$, *etc.*) of the program. Extend a state s to a logical T -interpretation $I : (D_I, \alpha_I)$: let D_I be all values of the program types; and construct α_I by using the assignments of s and adding assignments for all logical functions and predicates so that I is a T -interpretation. Subsequently, when we say a state, we mean either the assignment of program variables to values or the extension to a T -interpretation for the appropriate theory T , depending on the context. When we write $s \models F$, we mean that $I \models F$ for a T -interpretation I extending s .

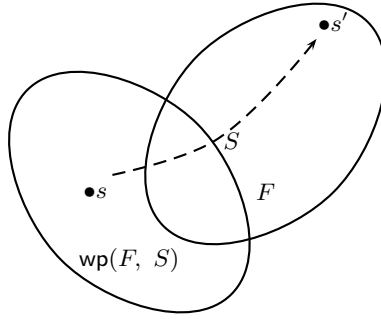


Fig. 5.22. Weakest precondition

Example 5.19. To extend

$$s : \{pc \mapsto L_1, a \mapsto [2; 0; 1], i \mapsto 2, j \mapsto 0, t \mapsto 2, rv \mapsto []\}$$

to a $(T_{\mathbb{Z}} \cup T_A)$ -interpretation $I : (D_I, \alpha_I)$, let D_I be the set of all integers and arrays of integers; and for α_I , assign the standard functions to $\cdot[\cdot]$, $\cdot\langle\cdot\cdot\rangle$, $+$, $-$, etc. ■

5.2.4 Verification Conditions

Our goal is to reduce an annotated function to a finite set of FOL formulae, called **verification conditions**, such that their validity implies that the function's behavior agrees with its annotations. The reduction to basic paths reduces reasoning about the function to reasoning about a finite set of basic paths. The final reduction from basic paths to VCs requires a mechanism for incorporating the effects of program statements into FOL formulae. The **weakest precondition** predicate transformer is the mechanism. A **predicate transformer** p is a function

$$p : \text{FOL} \times \text{stmts} \rightarrow \text{FOL}$$

that maps a FOL formula $F \in \text{FOL}$ and program statement $S \in \text{stmts}$ to a FOL formula.

The weakest precondition $\text{wp}(F, S)$ has the defining characteristic that if state s is such that

$$s \models \text{wp}(F, S)$$

and if statement S is executed on state s to produce state s' , then

$$s' \models F.$$

This situation is visualized in Figure 12.1(a). The region labeled F is the set of states that satisfy F ; similarly, the region labeled $\text{wp}(F, S)$ is the set of

states that satisfy $\text{wp}(F, S)$. Every state s on which executing statement S leads to a state s' in the F region must be in the $\text{wp}(F, S)$ region.

Define the weakest precondition for the two statement types of basic paths introduced in Section 5.2.1:

- *Assumption*: What must hold before statement **assume** c is executed to ensure that F holds afterward? If $c \rightarrow F$ holds before, then satisfying c in **assume** c guarantees that F holds afterward:

$$\text{wp}(F, \text{assume } c) \Leftrightarrow c \rightarrow F$$

- *Assignment*: What must hold before statement $v := e$ is executed to ensure that $F[v]$ holds afterward? If $F[e]$ holds before, then assigning e to v with $v := e$ makes $F[v]$ hold afterward:

$$\text{wp}(F[v], v := e) \Leftrightarrow F[e]$$

For a sequence of statements $S_1; \dots; S_n$, define

$$\text{wp}(F, S_1; \dots; S_n) \Leftrightarrow \text{wp}(\text{wp}(F, S_n), S_1; \dots; S_{n-1}) .$$

The weakest precondition moves a formula backward over a sequence of statements: for F to hold after executing $S_1; \dots; S_n$, $\text{wp}(F, S_1; \dots; S_n)$ must hold before executing the statements. Because basic paths have only assumption and assignment statements, the definition of wp is complete.

Then the **verification condition** of basic path

@ F
 S_1 ;
 $\vdots$
 S_n ;
 @ G

is

$$F \rightarrow \text{wp}(G, S_1; \dots; S_n) .$$

Its validity implies that when F holds before the statements of the path are executed, then G holds afterward. Traditionally, this verification condition is denoted by the **Hoare triple**

$$\{F\}S_1; \dots; S_n\{G\} .$$

Example 5.20. Consider the basic path

(1) ---

@ $x \geq 0$
 $x := x + 1$;
 @ $x \geq 1$

The VC is

$$\{x \geq 0\}x := x + 1\{x \geq 1\} : x \geq 0 \rightarrow \text{wp}(x \geq 1, x := x + 1)$$

so compute

$$\begin{aligned} & \text{wp}(x \geq 1, x := x + 1) \\ & \Leftrightarrow (x \geq 1)\{x \mapsto x + 1\} \\ & \Leftrightarrow x + 1 \geq 1 \\ & \Leftrightarrow x \geq 0 \end{aligned}$$

Simplifying the VC based on this computation produces

$$x \geq 0 \rightarrow x \geq 0,$$

which is clearly $T_{\mathbb{Z}}$ -valid. ■

Example 5.21. We generate the VC corresponding to basic path (2) in Example 5.13 (LinearSearch):

$$\begin{aligned} & \text{(2)} \\ @L : & F : \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e) \\ S_1 : & \text{assume } i \leq u; \\ S_2 : & \text{assume } a[i] = e; \\ S_3 : & rv := \text{true}; \\ @\text{post } G : & rv \leftrightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e \end{aligned}$$

The VC is

$$F \rightarrow \text{wp}(G, S_1; S_2; S_3),$$

so compute

$$\begin{aligned} & \text{wp}(G, S_1; S_2; S_3) \\ & \Leftrightarrow \text{wp}(\text{wp}(rv \leftrightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e, rv := \text{true}), S_1; S_2) \\ & \Leftrightarrow \text{wp}(\text{true} \leftrightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e, S_1; S_2) \\ & \Leftrightarrow \text{wp}(\exists j. \ell \leq j \leq u \wedge a[j] = e, S_1; S_2) \\ & \Leftrightarrow \text{wp}(\text{wp}(\exists j. \ell \leq j \leq u \wedge a[j] = e, \text{assume } a[i] = e), S_1) \\ & \Leftrightarrow \text{wp}(a[i] = e \rightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e, S_1) \\ & \Leftrightarrow \text{wp}(a[i] = e \rightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e, \text{assume } i \leq u) \\ & \Leftrightarrow i \leq u \rightarrow (a[i] = e \rightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e) \end{aligned}$$

Replacing F and $\text{wp}(G, S_1; S_2; S_3)$ according to the computed formula results in the VC

$$\begin{aligned} & \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e) \\ & \rightarrow (i \leq u \rightarrow (a[i] = e \rightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e)) \end{aligned}$$

or, equivalently,

$$\begin{aligned} & \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e) \wedge i \leq u \wedge a[i] = e \\ & \rightarrow \exists j. \ell \leq j \leq u \wedge a[j] = e \end{aligned}$$

according to the equivalence

$$F_1 \wedge F_2 \rightarrow (F_3 \rightarrow (F_4 \rightarrow F_5)) \Leftrightarrow (F_1 \wedge F_2 \wedge F_3 \wedge F_4) \rightarrow F_5 .$$

This formula is $(T_{\mathbb{Z}} \cup T_A)$ -valid. ■

Example 5.22. For basic path **(3)** of Example 5.13 (LinearSearch),

(3)

$$\begin{aligned} @L : F : & \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e) \\ S_1 : & \text{assume } i \leq u; \\ S_2 : & \text{assume } a[i] \neq e; \\ S_3 : & i := i + 1; \\ @L : G : & \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e) \end{aligned}$$

we use a different strategy to compute the corresponding VC

$$F \rightarrow \text{wp}(G, S_1; S_2; S_3) .$$

We collect all modifications to G during applications of wp and then apply them all at once. Compute

$$\begin{aligned} & \text{wp}(G, S_1; S_2; S_3) \\ \Leftrightarrow & \text{wp}(\text{wp}(G, i := i + 1), S_1; S_2) \\ \Leftrightarrow & \text{wp}(G\{i \mapsto i + 1\}, S_1; S_2) \\ \Leftrightarrow & \text{wp}(\text{wp}(G\{i \mapsto i + 1\}, \text{assume } a[i] \neq e), S_1) \\ \Leftrightarrow & \text{wp}(a[i] \neq e \rightarrow G\{i \mapsto i + 1\}, S_1) \\ \Leftrightarrow & \text{wp}(a[i] \neq e \rightarrow G\{i \mapsto i + 1\}, \text{assume } i \leq u) \\ \Leftrightarrow & i \leq u \rightarrow a[i] \neq e \rightarrow G\{i \mapsto i + 1\} \end{aligned}$$

Then the VC is

$$F \rightarrow i \leq u \rightarrow a[i] \neq e \rightarrow G\{i \mapsto i + 1\} ,$$

or

$$\begin{aligned} & \ell \leq i \wedge (\forall j. \ell \leq j < i \rightarrow a[j] \neq e) \wedge i \leq u \wedge a[i] \neq e \\ & \rightarrow \ell \leq i + 1 \wedge \forall j. \ell \leq j < i + 1 \rightarrow a[j] \neq e , \end{aligned}$$

which is $(T_{\mathbb{Z}} \cup T_A)$ -valid. ■

Example 5.23. Consider basic path **(2)** of Example 5.17 (BinarySearch):

(2)

$$\begin{aligned} @pre F : & 0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u) \\ S_1 : & \text{assume } \ell \leq u; \\ S_2 : & m := (\ell + u) \text{ div } 2; \\ S_3 : & \text{assume } a[m] = e; \\ S_4 : & rv := \text{true}; \\ @post G : & rv \leftrightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e \end{aligned}$$

The VC is

$$F \rightarrow \text{wp}(G, S_1; S_2; S_3; S_4),$$

so compute

$$\begin{aligned} & \text{wp}(G, S_1; S_2; S_3; S_4) \\ & \Leftrightarrow \text{wp}(\text{wp}(G, rv := \text{true}), S_1; S_2; S_3) \\ & \Leftrightarrow \text{wp}(G\{rv \mapsto \text{true}\}, S_1; S_2; S_3) \\ & \Leftrightarrow \text{wp}(\text{wp}(G\{rv \mapsto \text{true}\}, \text{assume } a[m] = e), S_1; S_2) \\ & \Leftrightarrow \text{wp}(a[m] = e \rightarrow G\{rv \mapsto \text{true}\}, S_1; S_2) \\ & \Leftrightarrow \text{wp}(\text{wp}(a[m] = e \rightarrow G\{rv \mapsto \text{true}\}, m := (\ell + u) \text{ div } 2), S_1) \\ & \Leftrightarrow \text{wp}((a[m] = e \rightarrow G\{rv \mapsto \text{true}\})\{m \mapsto (\ell + u) \text{ div } 2\}, S_1) \\ & \Leftrightarrow \text{wp}((a[m] = e \rightarrow G\{rv \mapsto \text{true}\})\{m \mapsto (\ell + u) \text{ div } 2\} \\ & \quad , \text{assume } \ell \leq u) \\ & \Leftrightarrow \ell \leq u \rightarrow (a[m] = e \rightarrow G\{rv \mapsto \text{true}\})\{m \mapsto (\ell + u) \text{ div } 2\} \end{aligned}$$

Applying the substitutions produces

$$\ell \leq u \rightarrow a[(\ell + u) \text{ div } 2] = e \rightarrow G\{rv \mapsto \text{true}, m \mapsto (\ell + u) \text{ div } 2\}.$$

Simplifying the VC accordingly

$$\begin{aligned} & 0 \leq \ell \wedge u < |a| \wedge \text{sorted}(a, \ell, u) \wedge \ell \leq u \wedge a[(\ell + u) \text{ div } 2] = e \\ & \rightarrow \exists i. \ell \leq i \leq u \wedge a[i] = e \end{aligned}$$

reveals that it is $(T_{\mathbb{Z}} \cup T_A)$ -valid: $\ell \leq u$ from the antecedent implies that $\ell \leq (\ell + u) \text{ div } 2 \leq u$, so that $a[(\ell + u) \text{ div } 2] = e$ implies that $\exists i. \ell \leq i \leq u \wedge a[i] = e$. ■

Example 5.24. Consider basic path **(3)** of Example 5.14 (BubbleSort):

(3)

$$\begin{aligned} @L_2 : F : & \left[\begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \\ \wedge \text{partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge \text{partitioned}(a, 0, j-1, j, j) \wedge \text{sorted}(a, i, |a|-1) \end{array} \right] \\ S_1 : & \text{assume } j < i; \\ S_2 : & \text{assume } a[j] > a[j+1]; \\ S_3 : & t := a[j]; \\ S_4 : & a[j] := a[j+1]; \\ S_5 : & a[j+1] := t; \\ S_6 : & j := j+1; \\ @L_2 : G : & \left[\begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \\ \wedge \text{partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge \text{partitioned}(a, 0, j-1, j, j) \wedge \text{sorted}(a, i, |a|-1) \end{array} \right] \end{aligned}$$

The VC is

$$F \rightarrow \text{wp}(G, S_1; S_2; S_3; S_4; S_5; S_6)$$

so compute

$$\begin{aligned}
& \text{wp}(G, S_1; S_2; S_3; S_4; S_5; S_6) \\
& \Leftrightarrow \text{wp}(\text{wp}(G, j := j + 1), S_1; S_2; S_3; S_4; S_5) \\
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1\}, S_1; S_2; S_3; S_4; S_5) \\
& \Leftrightarrow \text{wp}(\text{wp}(G\{j \mapsto j + 1\}, a[j + 1] := t), S_1; S_2; S_3; S_4) \\
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1\}\{a \mapsto a\langle j + 1 \triangleleft t \rangle\}, S_1; S_2; S_3; S_4)
\end{aligned}$$

Array assignment $a[j + 1] := t$ translates to the functional substitution $\{a \mapsto a\langle j + 1 \triangleleft t \rangle\}$ according to the theory of arrays, T_A . That is, the assignment is modeled by substituting $a\langle j + 1 \triangleleft t \rangle$ for every instance a affected by the assignment. Continuing,

$$\begin{aligned}
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j + 1 \triangleleft t \rangle\}, S_1; S_2; S_3; S_4) \\
& \Leftrightarrow \text{wp}(\text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j + 1 \triangleleft t \rangle\}, a[j] := a[j + 1]), \\
& \quad , S_1; S_2; S_3) \\
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j + 1 \triangleleft t \rangle\}\{a \mapsto a\langle j \triangleleft a[j + 1] \rangle\}, \\
& \quad , S_1; S_2; S_3) \\
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft t \rangle\}, S_1; S_2; S_3)
\end{aligned}$$

The array assignment S_4 similarly translates to a substitution. Composing the two substitutions produces the substitution in which the doubly-modified array $a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft t \rangle$ replaces a . Then

$$\begin{aligned}
& \Leftrightarrow \text{wp}(\text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft t \rangle\}, t := a[j]), \\
& \quad , S_1; S_2) \\
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft t \rangle\}\{t \mapsto a[j]\}, \\
& \quad , S_1; S_2) \\
& \Leftrightarrow \text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle\}, t \mapsto a[j]) \\
& \quad , S_1; S_2)
\end{aligned}$$

Here, the composition of substitutions results in applying $\{t \mapsto a[j]\}$ to the term $a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft t \rangle$, which contains t . To finish,

$$\begin{aligned}
& \Leftrightarrow \text{wp}(\text{wp}(G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle\}, t \mapsto a[j]) \\
& \quad , \text{assume } a[j] > a[j + 1]) \\
& \quad , S_1) \\
& \Leftrightarrow \text{wp}(a[j] > a[j + 1] \\
& \quad \rightarrow G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle\}, t \mapsto a[j]) \\
& \quad , S_1) \\
& \Leftrightarrow \text{wp}(a[j] > a[j + 1] \\
& \quad \rightarrow G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle\}, t \mapsto a[j]) \\
& \quad , \text{assume } j < i) \\
& \Leftrightarrow j < i \rightarrow a[j] > a[j + 1] \\
& \quad \rightarrow G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle\}, t \mapsto a[j]) \\
& \Leftrightarrow j < i \wedge a[j] > a[j + 1] \\
& \quad \rightarrow G\{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle\}, t \mapsto a[j])
\end{aligned}$$

Finally, applying the substitution

$$\sigma : \{j \mapsto j + 1, a \mapsto a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, t \mapsto a[j]\}$$

to G produces $G\sigma$:

$$\begin{aligned} & 1 \leq i < |a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle| \wedge 0 \leq j + 1 \leq i \\ & \wedge \text{partitioned}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, 0, i, i + 1, \\ & \quad |a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle| - 1) \\ & \wedge \text{partitioned}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, 0, j, j + 1, j + 1) \\ & \wedge \text{sorted}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, i, |a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle| - 1) \end{aligned}$$

The **length function** $|\cdot|$ obeys the following axiom:

$$\forall a, i, e. |a\langle i \triangleleft e \rangle| = |a| \quad (\text{array length})$$

In other words, modifying an array's elements does not change its size. Under this axiom, $G\sigma$ is equivalent to

$$\begin{aligned} & 1 \leq i < |a| \wedge 0 \leq j + 1 \leq i \\ & \wedge \text{partitioned}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, 0, i, i + 1, |a| - 1) \\ & \wedge \text{partitioned}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, 0, j, j + 1, j + 1) \\ & \wedge \text{sorted}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, i, |a| - 1) \end{aligned}$$

Thus, the VC is

$$\begin{aligned} & \left[\begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \\ \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \\ \wedge \text{partitioned}(a, 0, j - 1, j, j) \\ \wedge \text{sorted}(a, i, |a| - 1) \\ \wedge j < i \wedge a[j] > a[j + 1] \end{array} \right] \\ \rightarrow & \left[\begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j + 1 \leq i \\ \wedge \text{partitioned}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, 0, i, i + 1, |a| - 1) \\ \wedge \text{partitioned}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, 0, j, j + 1, j + 1) \\ \wedge \text{sorted}(a\langle j \triangleleft a[j + 1] \rangle\langle j + 1 \triangleleft a[j] \rangle, i, |a| - 1) \end{array} \right] \end{aligned}$$

which is $(T_{\mathbb{Z}} \cup T_A)$ -valid. ■

5.2.5 P -Invariant and P -Inductive

Let us consider the inductive assertion method abstractly.

A **program** is a set of functions with some distinguished **entry function**. Let P be a program with distinguished entry function f such that f has function precondition F_{pre} and initial location L_0 . A **computation** of program P , or a P -computation, is a sequence of states

$$s_0, s_1, s_2, \dots$$

such that

1. $s_0[pc] = L_0$ and $s_0 \models F_{\text{pre}}$;
2. and for each index i , s_{i+1} is a result of executing the instruction at $s_i[pc]$ on state s_i .

The notation $s_i[pc]$ indicates the value of pc given by state s_i . A computation may be finite or infinite. Infinite computations arise when a program contains a function that does not always halt (either through recursion or looping).

A formula F annotating location L of program P is **P -invariant** (also, is an invariant of P , or is a P -invariant) iff for all P -computations $s_0, s_1, s_2, \dots$ and for each index i , if $s_i[pc] = L$ then $s_i \models F$. The annotations of P are P -invariant iff each annotation is P -invariant. This definition is not implementable: checking if F is P -invariant requires checking an infinite number of P -computations in general, even if all computations are finite.

Instead, we use the inductive assertion method introduced in this chapter. If all verification conditions generated from program P are T -valid (in the proper theory T), then the annotations are **P -inductive** (also, are inductive P -invariants) and therefore P -invariant. In summary, we have the following theorem.

Theorem 5.25 (Verification Conditions). *If for every basic path*

$@L_i : F$
 $S_1;$
 $\vdots$
 $S_n;$
 $@L_j : G$

of program P , the verification condition

$$\{F\}S_1; \dots; S_n\{G\}$$

is valid (in the appropriate theory), then the annotations are P -invariant. In particular, the annotations are P -inductive.

In other words, when P 's annotations are P -inductive, each function of P obeys its specification.

Henceforth, when P is obvious from the context, we say invariant instead of P -invariant and inductive instead of P -inductive.

5.3 Total Correctness

Partial correctness is just one step in proving the **total correctness** of a function or program. Total correctness of a function asserts that if the input satisfies the function precondition, then the function *eventually halts* and produces output that satisfies its function postcondition. In other words, total

correctness requires proving partial correctness and that the function always halts on input satisfying its precondition. We focus now on the latter task.

Proving function termination is based on well-founded relations (see Chapter 4). Define a set S with a well-founded relation $\prec$. Then find a function δ mapping program states to S such that δ decreases according to $\prec$ along every basic path. Since $\prec$ is well-founded, there cannot exist an infinite sequence of program states; otherwise, they would map to an infinite decreasing sequence in S . The function δ is called a **ranking function**.

Choosing Well-Founded Relations and Ranking Functions

The concept of well-founded relations is the fundamental principle of this method. The ranking function itself is really just a convenience. One could directly construct a well-founded relation over the program states and show that the output state of each basic path is less than the input state according to this relation. However, it is often conceptually easier to find a function that maps states to a known set S with a known well-founded relation $\prec$. For example, we usually choose as S the set of n -tuples of natural numbers and as $\prec$ the lexicographic extension $<_n$ of $<$ to n -tuples of natural numbers, where n varies according to the application.

As with partial correctness, we consider total correctness in the context of loops first and then in the context of recursion. Section 6.2 examines a program with both loops and recursion.

Example 5.26. Figure 5.23 lists BubbleSort with ranking annotations. It contains one new type of annotation: $\downarrow (i+1, i+1)$ and $\downarrow (i+1, i-j)$ assert that the functions $(i+1, i+1)$ and $(i+1, i-j)$, respectively, are ranking functions. These functions map states of BubbleSort onto pairs of natural numbers $S : \mathbb{N}^2$ with well-founded relation $<_2$. Intuitively, we have captured two separate arguments. The outer loop eventually finishes because i decreases to 0; hence $i+1$ decreases as well. Why do we use $i+1$ rather than i ? When $|a| = 0$, the initial assignment to i is -1 . While $i+1$ is always nonnegative, i is not; and recall that we want to map into the natural numbers. The inner loop halts because j increases to i ; hence $i-j$ decreases to 0. Therefore, our intuition tells us that $i+1$ is important for the outer loop, and $i-j$ is important for the inner loop.

Placing these two functions $i+1$ and $i-j$ together as a pair $(i+1, i-j)$ provides the annotation for the inner loop. We expect $i+1$ to remain constant while the inner loop is executing and decreasing $i-j$. For the annotation of the outer loop, we note that $i+1 > i-j = i-0 = i$ on entry to the inner loop, so that $(i+1, i+1) >_2 (i+1, i-j)$.

The loop annotations assert that the ranking functions $(i+1, i+1)$ and $(i+1, i-j)$ map program states to pairs of natural numbers. Hence, we need to prove that the loop annotations are inductive using the inductive assertion

```

@pre  $\top$ 
@post  $\top$ 
int[] BubbleSort(int[]  $a_0$ ) {
    int[]  $a := a_0$ ;
    for
        @ $L_1 : i + 1 \geq 0$ 
        ↓ ( $i + 1, i + 1$ )
        (int  $i := |a| - 1$ ;  $i > 0$ ;  $i := i - 1$ ) {
            for
                @ $L_2 : i + 1 \geq 0 \wedge i - j \geq 0$ 
                ↓ ( $i + 1, i - j$ )
                (int  $j := 0$ ;  $j < i$ ;  $j := j + 1$ ) {
                    if ( $a[j] > a[j + 1]$ ) {
                        int  $t := a[j]$ ;
                         $a[j] := a[j + 1]$ ;
                         $a[j + 1] := t$ ;
                    }
                }
            }
        }
    return  $a$ ;
}

```

Fig. 5.23. BubbleSort with annotations to prove termination

method. We leave this step to the reader. It only remains to prove that the functions decrease along each basic path.

The relevant basic paths are the following:

(1)

```

@ $L_1 : i + 1 \geq 0$ 
↓  $L_1 : (i + 1, i + 1)$ 
assume  $i > 0$ ;
 $j := 0$ ;
↓  $L_2 : (i + 1, i - j)$ 

```

(2)

```

@ $L_2 : i + 1 \geq 0 \wedge i - j \geq 0$ 
↓  $L_2 : (i + 1, i - j)$ 
assume  $j < i$ ;
assume  $a[j] > a[j + 1]$ ;
 $t := a[j]$ ;
 $a[j] := a[j + 1]$ ;
 $a[j + 1] := t$ ;
 $j := j + 1$ ;
↓  $L_2 : (i + 1, i - j)$ 

```

(3)

```

@L2 : i + 1 ≥ 0 ∧ i - j ≥ 0
↓ L2 : (i + 1, i - j)
assume j < i;
assume a[j] ≤ a[j + 1];
j := j + 1;
↓ L2 : (i + 1, i - j)

```

(4)

```

@L2 : i + 1 ≥ 0 ∧ i - j ≥ 0
↓ L2 : (i + 1, i - j)
assume j ≥ i;
i := i - 1;
↓ L1 : (i + 1, i + 1)

```

The paths entering and exiting the outer loop at L_1 are not relevant for the termination argument. The entering path does not begin with a ranking function annotation, so there is nothing to prove. The exiting path leads to the **return** statement.

For termination purposes, paths (2) and (3) can be treated the same:

```

@L2 : i + 1 ≥ 0 ∧ i - j ≥ 0
↓ L2 : (i + 1, i - j)
assume j < i;
...
j := j + 1;
↓ L2 : (i + 1, i - j)

```

The excluded statements do not impact the value of the ranking functions. ■

Verification Conditions

A basic path beginning and ending with a ranking function

```

@ F
↓ δ[ $\overline{x}$ ]
S1;
⋮
Sk;
↓ κ[ $\overline{x}$ ]

```

induces a verification condition

$$F \rightarrow \text{wp}(\kappa \prec \delta[\overline{x}_0], S_1; \dots; S_k) \{ \overline{x}_0 \mapsto \overline{x} \} .$$

In words, we must prove that the value of κ after executing statements $S_1; \dots; S_k$ is less than the value of δ before executing the statements. Therefore, we rename the variables of δ to something new ($\overline{x}_0$ in this case), compute the weakest precondition of $\kappa \prec \delta[\overline{x}_0]$ across the statements $S_1; \dots; S_k$, and then rename the new variables back to their original names ($\overline{x}$ in this case). This renaming process preserves the value of $\delta[\overline{x}]$ in its context. The annotation F can provide extra invariant information with which to prove this relation.

Example 5.27. Let us return to the proof of Example 5.26 that **BubbleSort** halts. Path (1) induces the following verification condition:

$$i + 1 \geq 0 \wedge i > 0 \rightarrow (i + 1, i - 0) <_2 (i + 1, i + 1) ,$$

which is valid. Paths (2) and (3) induce the verification condition:

$$i + 1 \geq 0 \wedge i - j \geq 0 \wedge j < i \rightarrow (i + 1, i - (j + 1)) <_2 (i + 1, i - j) ,$$

which is valid. Finally, (4) induces the verification condition

$$i + 1 \geq 0 \wedge i - j \geq 0 \wedge j \geq i \rightarrow ((i - 1) + 1, (i - 1) + 1) <_2 (i + 1, i - j) ,$$

which is also valid. Hence, **BubbleSort** always halts. Combined with the proof of the sortedness property, we can now say that **BubbleSort** is totally correct with respect to its specification: it always halts and returns a sorted array.

Let us work through the construction of the final verification condition for basic path (4). First, replace i and j with i_0 and j_0 , respectively, in the function annotating L_2 : $(i_0 + 1, i_0 - j_0)$. Then compute

$$\begin{aligned} & \text{wp}((i + 1, i + 1) <_2 (i_0 + 1, i_0 - j_0), \text{assume } j \geq i; i := i - 1) \\ & \Leftrightarrow \text{wp}(((i - 1) + 1, (i - 1) + 1) <_2 (i_0 + 1, i_0 - j_0), \text{assume } j \geq i) \\ & \Leftrightarrow j \geq i \rightarrow (i, i) <_2 (i_0 + 1, i_0 - j_0) \end{aligned}$$

Now, having preserved the original value of $(i + 1, i - j)$ at L_2 , replace i_0 and j_0 with i and j , respectively:

$$j \geq i \rightarrow (i, i) <_2 (i + 1, i - j) .$$

Noting that (4) begins by asserting $i + 1 \geq 0 \wedge i - j \geq 0$, we have our verification condition:

$$i + 1 \geq 0 \wedge i - j \geq 0 \wedge j \geq i \rightarrow (i, i) <_2 (i + 1, i - j) .$$

In this proof, the loop annotations (other than the ranking functions) do not have any bearing on the termination argument. Their purpose is only to prove that the given functions map to the natural numbers. ■

```

@pre  $u - \ell + 1 \geq 0$ 
@post  $\top$ 
↓  $u - \ell + 1$ 
bool BinarySearch(int[] a, int  $\ell$ , int  $u$ , int  $e$ ) {
    if ( $\ell > u$ ) return false;
    else {
        int  $m := (\ell + u) \text{ div } 2$ ;
        if ( $a[m] = e$ ) return true;
        else if ( $a[m] < e$ ) return BinarySearch( $a, m + 1, u, e$ );
        else return BinarySearch( $a, \ell, m - 1, e$ );
    }
}

```

Fig. 5.24. BinarySearch always halts

Besides loops, recursion is another source of nontermination and hence requires ranking annotations as well. Again, we break the termination argument down to the level of basic paths.

Example 5.28. Figure 5.24 lists the BinarySearch function. $u - \ell + 1$ maps the formal parameters of BinarySearch to $S : \mathbb{N}$ with well-founded relation $<$. Why do we use $u - \ell + 1$ as the ranking function? Intuitively, the interval $[\ell, u]$ shortens at each level of recursion, so $u - \ell$ is a good guess at a ranking function. However, it may be that $\ell > u$ so that $u - \ell$ does not map into $\mathbb{N}$; but in this case $\ell = u + 1$, so $u - \ell + 1$ does map into $\mathbb{N}$.

We now need to prove two properties of $u - \ell + 1$:

1. Because $u - \ell + 1$ has type `int`, we must prove that $u - \ell + 1$ in fact maps to $\mathbb{N}$: whenever $u - \ell + 1$ is evaluated at function entry, it must be the case that $u - \ell + 1 \geq 0$.
2. We must prove that $u - \ell + 1$ decreases from function entry to each recursive call.

The function precondition asserts the first property, so we need only prove that the function specification is inductive as usual. To prove the second property, we reduce the argument to basic paths: $u - \ell + 1$ should decrease across each basic path.

Convince yourself that the annotations other than the ranking argument are inductive.

We now consider the ranking argument. The relevant basic paths of the function are the following:

(1)

```

@pre  $u - \ell + 1 \geq 0$ 
↓  $u - \ell + 1$ 
assume  $\ell \leq u$ ;
 $m := (\ell + u) \text{ div } 2$ ;
assume  $a[m] \neq e$ ;
assume  $a[m] < e$ ;
↓  $u - (m + 1) + 1$ 

```

(2)

```

@pre  $u - \ell + 1 \geq 0$ 
↓  $u - \ell + 1$ 
assume  $\ell \leq u$ ;
 $m := (\ell + u) \text{ div } 2$ ;
assume  $a[m] \neq e$ ;
assume  $a[m] \geq e$ ;
↓  $(m - 1) - \ell + 1$ 

```

Two other basic paths exist from function entry to the first two **return** statements; however, as the recursion ends at each, they are irrelevant to the termination argument.

The basic paths induce two verification conditions. Before examining them, notice that the **assume** statements about $a[m]$ are irrelevant to the termination argument. Now, the first VC is

$$\begin{aligned}
 &u - \ell + 1 \geq 0 \wedge \ell \leq u \wedge \dots \\
 &\rightarrow u - (((\ell + u) \text{ div } 2) + 1) + 1 < u - \ell + 1,
 \end{aligned}$$

where $\dots$ elides the literals involving $a[m]$. It is $T_{\mathbb{Z}}$ -valid. The VC

$$\begin{aligned}
 &u - \ell + 1 \geq 0 \wedge \ell \leq u \wedge \dots \\
 &\rightarrow (((\ell + u) \text{ div } 2) - 1) - \ell + 1 < u - \ell + 1
 \end{aligned}$$

for the second basic path is also $T_{\mathbb{Z}}$ -valid, so **BinarySearch** halts on all input in which ℓ is initially at most $u + 1$.

Section 6.2 provides an alternative to using the awkward ranking function $u - \ell + 1$. Additionally, the argument proves termination on all input. ■

5.4 Summary

This chapter introduces the specification and verification of sequential programs. It covers:

- The programming language **pi**.

- *Program specification.* Specifying a program involves writing *function preconditions* and *function postconditions* as first-order assertions. A function precondition asserts on which inputs the function is defined, while a function postcondition asserts the form of the returned data. Other annotations: loop invariants, assertions, runtime assertions.
- *Partial correctness*, which guarantees that if a function halts and the input satisfied the function precondition, then its returned value satisfies the function postcondition. Partial correctness is proved via an inductive argument. Additional annotations strengthen the inductive hypothesis. Basic paths, program state, verification conditions, inductive invariants.
- *Total correctness*, which guarantees additionally that the program or function always halts. Total correctness requires mapping, via a ranking function, program states to a domain with a well-founded relation and proving that the mapped values in the domain decrease as computation progresses. Typically, proving termination requires additional partial correctness annotations.

Chapter 6 discusses strategies for applying the techniques of this chapter.

The methods introduced in this chapter are fundamental to verification of software and hardware. However, we present them in a simple context. Decades of research have made much more possible.

The validity of verification conditions listed in examples or produced by programs in the chapter are all decided using the decision procedures discussed in Part II. We have focused on specifications that can be expressed in the fragments of theories introduced in Chapter 3. More complex specifications may require general mechanical theorem proving. However, mechanical theorem provers often rely on the decision procedures of Part II when possible for speed and to minimize human interaction.

Chapter 12 introduces algorithms for deducing annotations.

Bibliographic Remarks

Formally proving program correctness has been a subject of active research for five decades. McCarthy argues in [59, 58] for a “mathematical science of computation”. Floyd [34] and Hoare [39] introduce the main concepts for proving property invariance and termination. In particular, they develop *Floyd-Hoare logic*. Manna describes a verification style similar to ours [52]. The *weakest precondition* predicate transformer was first formalized by Dijkstra [28].

King describes in his thesis [50] the idea of a *verifying compiler*, which generates and proves during compilation the verification conditions that arise from program annotations. See [27] for a discussion of the *Extended Static Checker*, a verifying compiler for Java.

```

@pre p(a0)
@post sorted(rv, 0, |rv| - 1)
int[] InsertionSort(int[] a0) {
  int[] a := a0;
  for
    @ r1(a, a0, i, j)
    (int i := 1; i < |a|; i := i + 1) {
      int t := a[i];
      for
        @ r2(a, a0, i, j)
        (int j := i - 1; j ≥ 0; j := j - 1) {
          if (a[j] ≤ t) break;
          a[j + 1] := a[j];
        }
      a[j + 1] := t;
    }
  return a;
}

```

Fig. 5.25. InsertionSort for Exercise 5.1(a)

Exercises

5.1 (Basic paths). For each of the following functions, replace each @pre $\top$ with a fresh predicate p over the function parameters, each @post $\top$ with a fresh predicate q over rv and the function parameters, and each @ $\top$ with a fresh predicate r over the function variables. As an example, see Figure 5.25 for the replacements for part (a). Then list the basic paths.

- (a) InsertionSort of Figure 6.8.
- (b) merge of Figure 6.9.
- (c) ms of Figure 6.9.

5.2 (Weakest precondition). Compute the following formulae:

- (a) $\text{wp}(x \geq 0, x := x - k; \text{assume } k \leq 1)$
- (b) $\text{wp}(x \geq 0, \text{assume } k \leq x; x := x - k)$
- (c) $\text{wp}(x \geq 0, x := x - k; \text{assume } k \leq x)$
- (d) $\text{wp}(x + 2y \geq 3, x := x + 1; \text{assume } x > 0; y := y + x)$

5.3 (Verification condition generation). Generate the VCs for the following basic paths:

(1)

```

@ x > 0;
x := x - k;
assume k ≤ 1;
@ x ≥ 0;

```

(2)

```
@ T;
assume k ≤ x;
x := x - k;
@ x ≥ 0;
```

(3)

```
@ T;
x := x - k;
assume k ≤ x;
@ x ≥ 0;
```

(4)

```
@ k ≥ 0;
x := x - k;
assume k ≤ x;
@ x ≥ 0;
```

(5)

```
@ y ≥ 0;
x := x + 1;
assume x > 0;
y := y + x;
@ x + 2y ≥ 3;
```

Which are $T_{\mathbb{Z}}$ -valid? Which are $T_{\mathbb{Q}}$ -valid?

5.4 (Verification conditions). For each basic path generated in Exercise 5.1, list the corresponding VCs.

5.5 (Public functions). Example 5.6 asserts that a function that is accessible outside a module should have a reasonable function precondition, such as $\top$. Implement verified public wrapper functions to `LinearSearch` and `BinarySearch` for searching an entire array. The function preconditions should be reasonable.

5.6 (The `div` function). Integer division, even by a constant, is not a function of $T_{\mathbb{Z}}$; however, it is useful for reasoning about programs like `BinarySearch`. Show how basic paths that include the `div` function can be altered to use only standard linear arithmetic. *Hint:* Use an additional `assume` statement. How does this change affect the resulting VCs?

5.7 (Ackermann function). Implement the *ack* function of Example 4.9 as a `pi` program and prove that it always halts.